

Infografía

Módulo 10 — Shaders · Sesión 2

**Vestir sprites: texturas, distorsión, vertex y el puente con
GDScript**

UPB · Gestión I/2026

Plan de la clase

1. **Leer la textura** — gris, sepia, tinte (escena 07).
2. **Mirar a los vecinos** — el contorno (escena 07).
3. **Distorsión** — cambiar *dónde* lees (escena 08).
4. **vertex()** — mover geometría, no color (escena 09).
5. **El puente** — GDScript escribe uniforms (escena 10, en vivo).
6. **El gotcha** — el material compartido.
7. **Ejercicio** — disolver al enemigo (escena 11).

*Recap de 20 segundos: Material → Nuevo ShaderMaterial → Shader → Nuevo Shader → Ctrl+S y se ve.
Hoy el lienzo ya no está vacío: es tu sprite.*

La pregunta del día

Ayer pintamos de la nada: matemática sobre un rectángulo.

Pero tu juego ya tiene sprites.
¿Cómo se ponen grises cuando mueren,
blancos cuando reciben daño?

Mismo shader de ayer + una entrada nueva: la imagen del sprite.

Leer la textura (escena 07)

```
vec4 original = texture(TEXTURE, UV);    // el color DE ESTE PIXEL en la imagen

// luminancia: qué tan claro se ve un color (el ojo pesa más el verde)
float lum = dot(original.rgb, vec3(0.299, 0.587, 0.114));

COLOR = vec4(mix(original.rgb, vec3(lum), fuerza), original.a);
```

- `TEXTURE` = la imagen del sprite · `texture(...)` = leerla en una posición.
- gris / sepia / tinte: **matemática sobre el color leído.**

Perilla: `fuerza` va de 0 a 1 — el sprite se desatura en vivo. Conserva el alfa: `original.a`.

¿Qué le pasa a este sprite? • 1

```
void fragment() {  
    vec4 c = texture(TEXTURE, UV);  
    COLOR = vec4(c.rgb * vec3(0.0, 1.0, 0.0), c.a);  
}
```

- A. Solo sobrevive el canal verde: lo rojo/azul queda negro
- B. El sprite entero se pinta de verde sólido
- C. Se vuelve más brillante
- D. No cambia nada

Multiplicar por 0 mata el canal; por 1 lo deja pasar.

Mirar a los vecinos: el contorno (escena 07)

```
vec2 paso = TEXTURE_PIXEL_SIZE * grosor;    // cuánto mide "un pixel" en UV
float alfa_arriba = texture(TEXTURE, UV + vec2(0.0, -paso.y)).a;
// ...abajo, izquierda, derecha

// si YO soy transparente pero un vecino es opaco → estoy en el borde
if (soy_transparente && vecino_opaco) { COLOR = color_contorno; }
```

Un pixel puede leer la textura DONDE QUIERA — no solo en su UV. Eso abre: contornos, blur, distorsión...

Distorsión: no cambies el color, cambia DÓNDE lees (escena 08)

```
// agua: una onda radial que viaja con TIME desplaza las UV
float onda = sin(dist * frecuencia - TIME * velocidad);
COLOR = texture(TEXTURE, UV + vec2(onda) * magnitud);

// pixelar: floor otra vez - toda la celda lee el MISMO punto
vec2 uv_grilla = floor(UV / tamano_pixel) * tamano_pixel;
COLOR = texture(TEXTURE, uv_grilla);
```

TIME es el reloj del shader: la onda viaja sola, sin _process ni señales. Perilla: sube `magnitud` hasta que se rompa feo — y ya sabes qué controla.

¿Qué le pasa a este sprite? • 2

```
void fragment() {  
    vec2 uv = UV;  
    uv.y += sin(uv.x * 20.0 + TIME * 3.0) * 0.02;  
    COLOR = texture(TEXTURE, uv);  
}
```

- A. Ondas verticales que recorren el sprite (bandera)
- B. Filas que se corren a izquierda/derecha (calor)
- C. El sprite entero rebota arriba y abajo
- D. Zoom que late

¿Qué desplazo (y) y según qué varía (x)?

vertex() : el otro programa (escena 09)

```
// corre por VÉRTICE, antes
// que fragment(); mueve
// GEOMETRÍA (píxeles locales)
void vertex() {
    VERTEX.y += sin(TIME * vel
        + VERTEX.x * frec) * amp;
}
```

El detalle que importa:

un `Sprite2D` es un quad: 4 vértices.

Este shader solo puede mover esquinas
→ el sprite *se tambalea*.

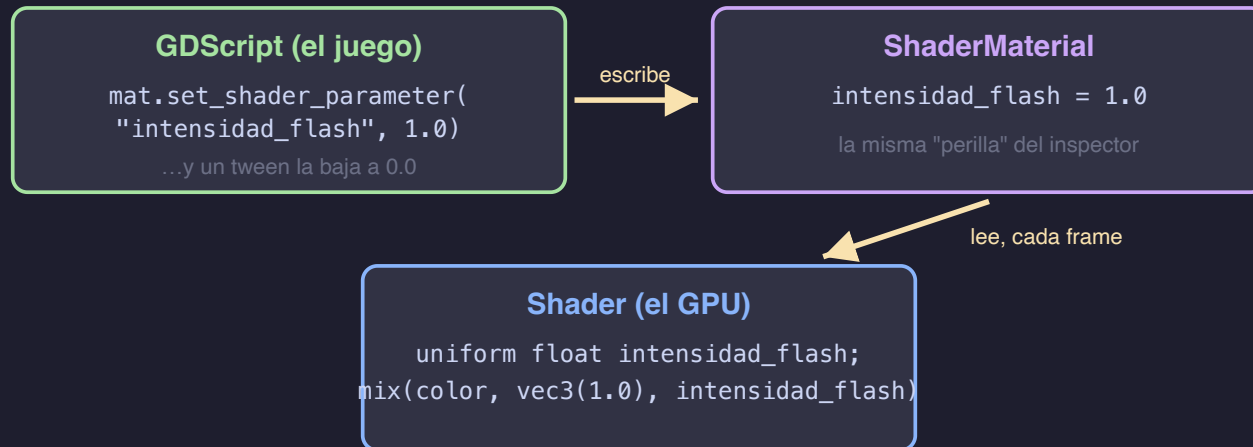
Una malla subdividida (`MeshInstance2D`

- `PlaneMesh`) tiene cientos
→ la misma fórmula *ondea suave*.

En la escena 09 están lado a lado, con el MISMO shader. El pasto: el desplazamiento se multiplica por la altura — la base no se mueve.

El puente: GDScript escribe uniforms (escena 10 • 🛠️ en vivo)

El puente: el juego escribe, el shader lee



un uniform es la única puerta entre tu juego y el shader — el inspector y GDScript tocan la MISMA perilla

El puente, en código

```
# scripts/hit_flash.gd - al pulsar H:  
var mat := material as ShaderMaterial  
mat.set_shader_parameter("intensidad_flash", 1.0)  
var tween := create_tween()  
tween.tween_property(mat, "shader_parameter/intensidad_flash", 0.0, 0.3)
```

```
// hit_flash.gdshader - falta UNA línea (la escribimos ahora):  
uniform float intensidad_flash : hint_range(0.0, 1.0) = 0.0;  
// TODO (en vivo): original.rgb = mix(original.rgb, vec3(1.0), intensidad_flash);
```

El script ya empuja la perilla. Por eso H "no hace nada": el shader la ignora. Una línea y aparece el hit flash de todos los juegos.

El gotcha: el material se COMPARTE

Escribimos la línea, pulsamos H y... **flashean los DOS sprites.**

- Un recurso (`ShaderMaterial`) es **compartido por defecto**: ambos sprites apuntan al mismo material → misma perilla.
- Arreglos:
 - **Local to Scene** en el material (`resource_local_to_scene`), o
 - duplicarlo por instancia: `material = material.duplicate()` en `_ready()`.

Este es EL bug clásico de shaders en juegos reales: "le pego a un enemigo y parpadean todos". Ahora ya lo viste venir.

¿Qué le pasa a este sprite? • 3

```
void fragment() {  
    vec4 c = texture(TEXTURE, UV);  
    if (c.a < 0.5) { discard; }  
    COLOR = c;  
}
```

- A. Los bordes semitransparentes desaparecen (alfa "duro")
- B. El sprite entero se vuelve invisible
- C. No cambia nada visible
- D. El sprite se pinta a mitad de opacidad

discard = este pixel no se dibuja. Es la pieza del ejercicio.

🎓 Ejercicio: disolver al enemigo (escena 11)

Haz clic... ¿no pasa nada? **Falta tu línea.** Todo lo demás ya está cableado:

- `ruido_tex`: una `NoiseTexture2D` (nubes 0–1) asignada en el material.
- `progreso`: el clic lo anima 0 → 1 con un tween (¡el puente de hoy!).

La idea: a medida que `progreso` sube, los pixeles cuyo ruido quedó **por debajo** ya se quemaron → `discard`.

Tres pistas en `shaders/exercises/disolver.gdshader`. Pregunta venenosa: ¿qué pasa si inviertes la comparación? Pruébalo y explícalo. Reto extra: borde de fuego.

Resumen – la chuleta de hoy

Pieza	Qué hace
<code>texture(TEXTURE, UV)</code>	leer la imagen del sprite en una posición
<code>TEXTURE_PIXEL_SIZE</code>	el paso exacto para mirar al pixel vecino
distorsión	cambiar dónde lees, no qué color pones
<code>vertex()</code> / <code>VERTEX</code>	mover geometría; sprite = solo 4 vértices
<code>set_shader_parameter</code>	GDScript escribe la perilla del shader
<code>tween_property(mat, "shader_parameter/...")</code>	animarla
<code>discard</code>	este pixel no se dibuja
material compartido	<code>resource_local_to_scene</code> o <code>duplicate()</code>

La próxima: el "sprite" será la PANTALLA ENTERA – y la luz 2D con normal maps.